COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	04 / 08 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

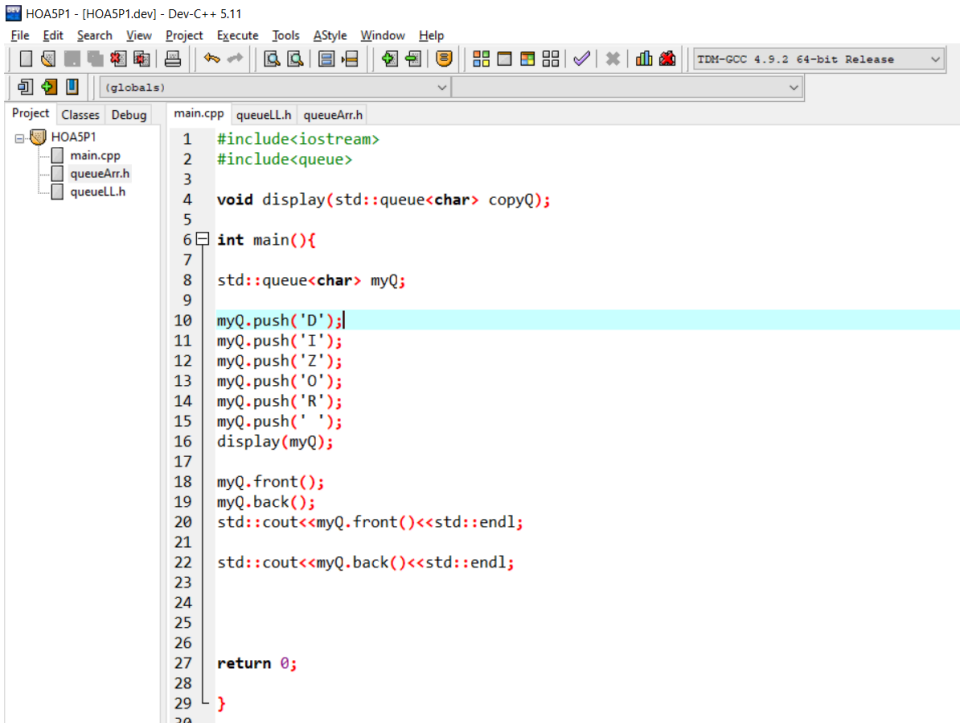
ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

CODE



```
30
31  void display(std::queue<char> copyQ){
32
33      std::queue<char> c = copyQ;
34
35      while  (!c.empty()){
36          std::cout<<" " <<c.front();
37          c.pop();
38      }
39
40      std::cout<<"\n";
41
42  }
43
```

OUTPUT

```
Select C:\Users\elija\OneDrive\Desktop\HOA5P1\HOA5P1.exe
 D I Z O R
D


--------------------------------
Process exited after 0.02218 seconds with return value 0
Press any key to continue . . .
```

OBSERVATION

It creates a standard template library to manage sequential char based containers. It populates the queue with character elements to spell out a name sequence using the .push().. The 'display' prints out the queue contents by receiving a copy of the container by value and calling .front() and .pop() until it is empty. After that, the program verifies the boundaries of the queue by executing .front() and .back() calls to access and print the head and tail.

ILO B

COMPUTER ENGINEERING

QUEZON CITY

CODE

ILOB - [ILOB.dev] - [Executing] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project Classes Debug

ILOB
- main.cpp
- queueArr.h
- queueLL.h

main.cpp queueLL.h queueArr.h

```cpp
#include <iostream>
#include "queueLL.h"

int main(){

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('E', &front, &back);
    std::cout<<  front->data<<""<<back->data<<std::endl;
    enqueue('L', &front, &back);
    std::cout<<  front->data<<""<<back->data<<std::endl;
    enqueue('I', &front, &back);

    std::cout<<  front->data<<""<<back->data<<std::endl;

    dequeue(&front, &back);
    std::cout<< front->data <<""<< back->data<<std::endl;

    return 0;
}
```

ILOB - [ILOB.dev] - [Executing] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project Classes Debug

ILOB
- main.cpp
- queueArr.h
- queueLL.h

main.cpp queueLL.h queueArr.h

```cpp
#ifndef QUEUELL_H
#define QUEUEULL_H

template <typename T>
class qNode{
    public:
    T data;
    qNode<T>* next;
};

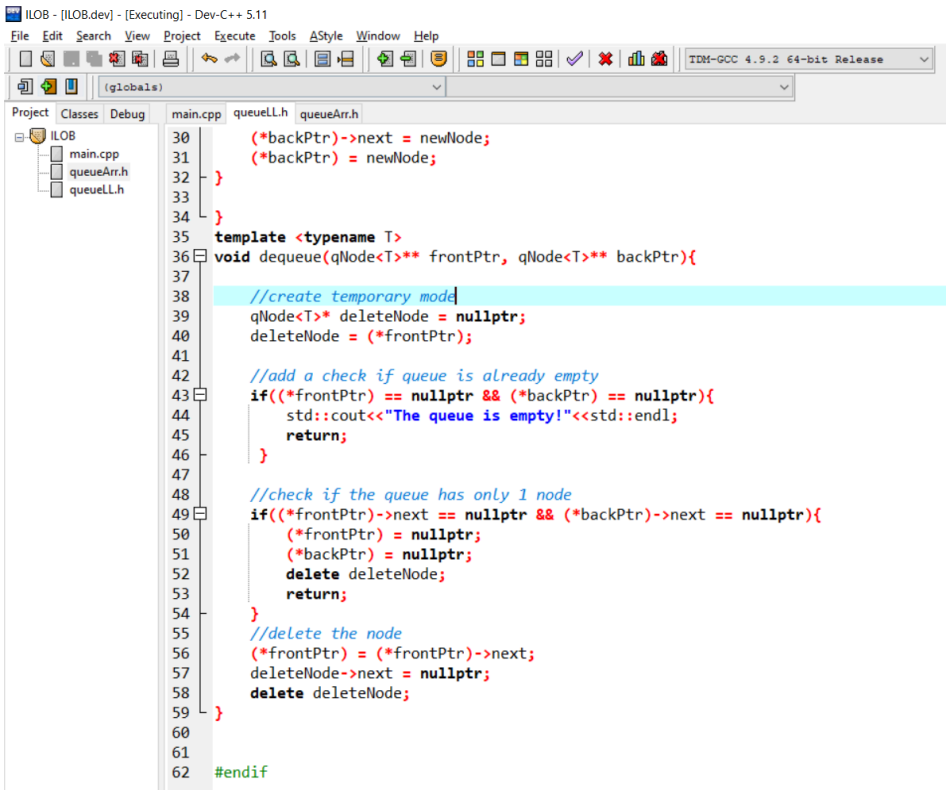
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}


template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){

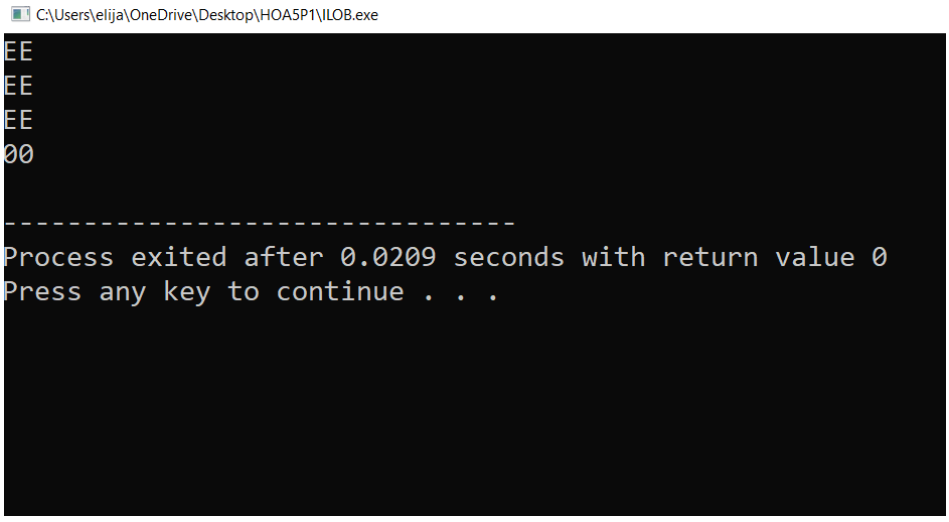
qNode<T>* newNode = new_node(newData);


if ((*frontPtr)==nullptr && (*backPtr) == nullptr){
    (*frontPtr) = newNode;
    (*backPtr) = newNode;

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```



OUTPUT



OBSERVATION

The program uses a template class to create a queue with a linked list, as the enqueue adds items to the back, the dequeue removes them from the front while cleaning up the used memory.

ILO B modded (displayAll,checkfront&back)l

CODE

ILOB - [ILOB.dev] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project Classes Debug

ILOB
ILOB modded.cp
queueLL.h

ILOB modded.cpp queueLL.h

```cpp
#include <iostream>
#include "queueLL.h"

int main() {
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('E', &front, &back);
    checkfrontandback(front, back);
    displayAll(front);

    enqueue('L', &front, &back);
    checkfrontandback(front, back);
    displayAll(front);

    enqueue('I', &front, &back);
    checkfrontandback(front, back);
    displayAll(front);

    dequeue(&front, &back);
    checkfrontandback(front, back);
    displayAll(front);

    return 0;
```

ILOB - [ILOB.dev] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project Classes Debug

ILOB
ILOB modded.cp
queueLL.h

ILOB modded.cpp queueLL.h

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

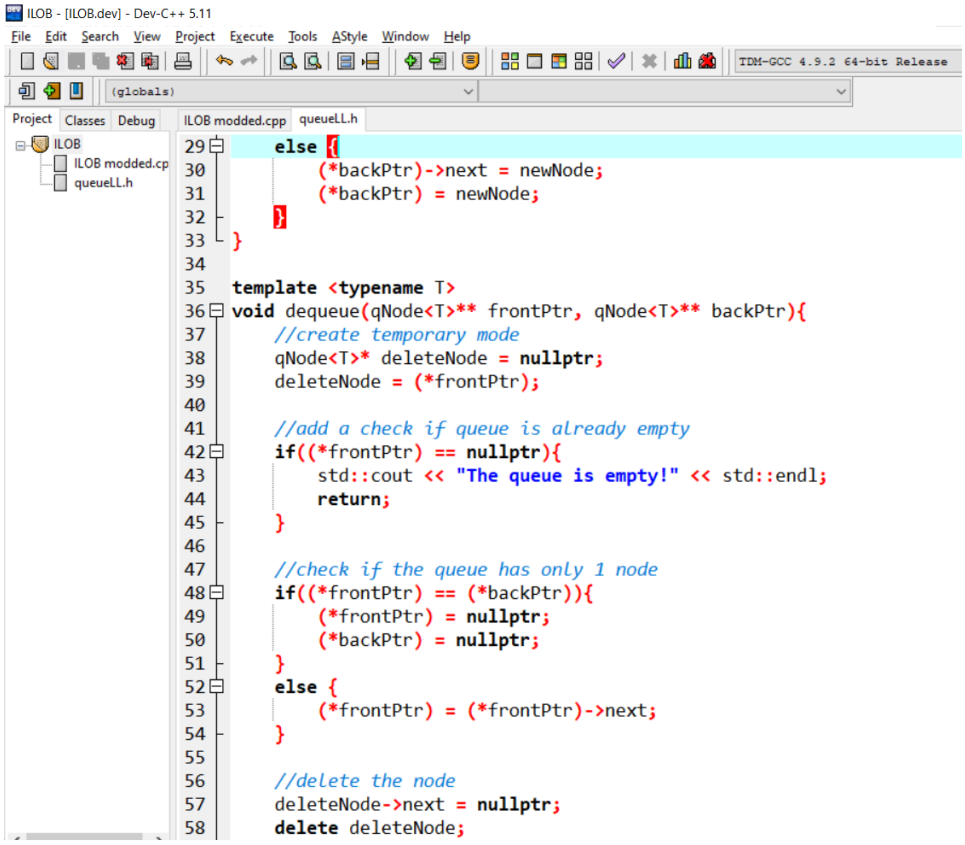
#include <iostream>

template <typename T>
class qNode{
    public:
    T data;
    qNode<T>* next;
};

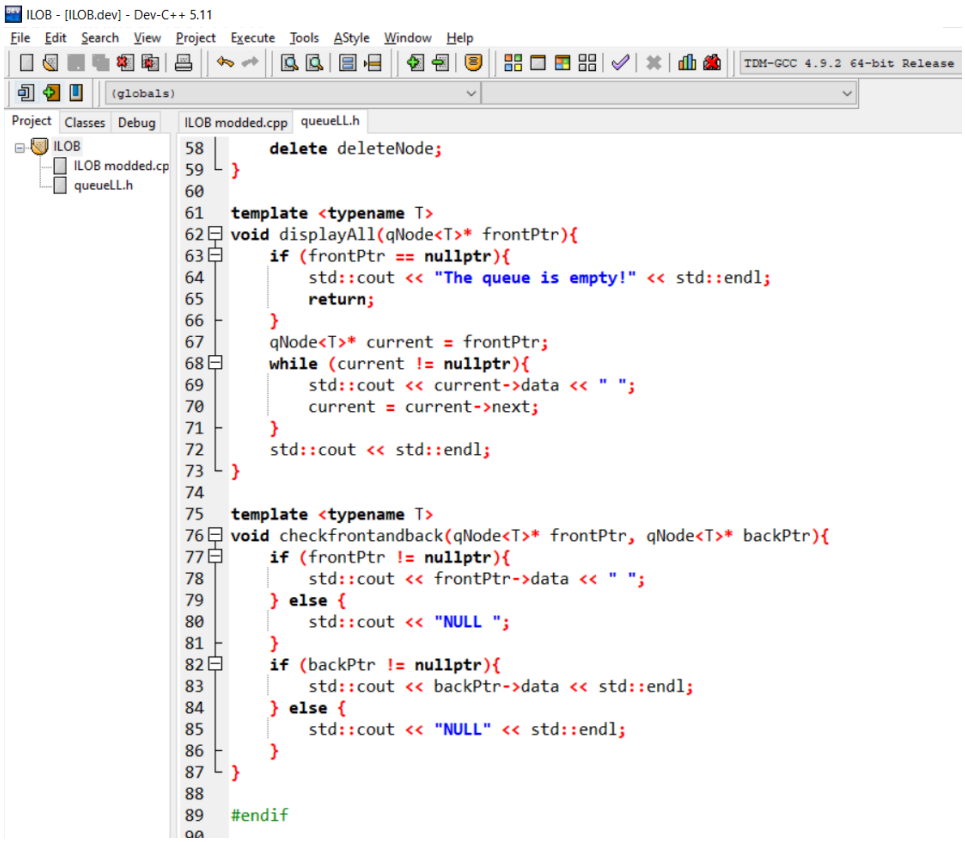
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

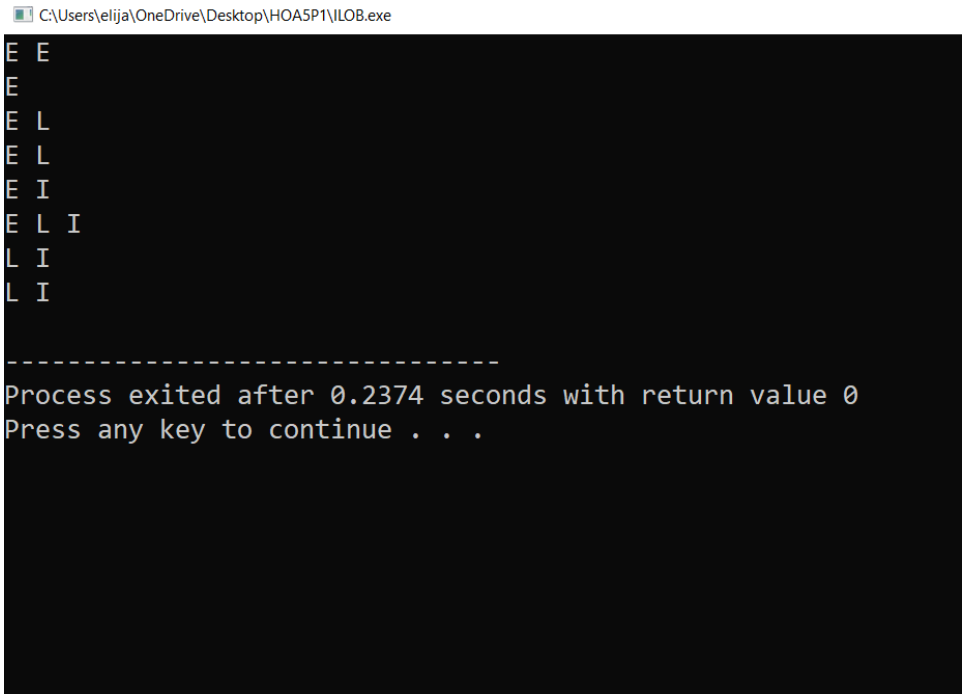
template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* newNode = new_node(newData);

    if ((*frontPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
```




```
29      else {
30          (*backPtr)->next = newNode;
31          (*backPtr) = newNode;
32      }
33  }
34
35  template <typename T>
36  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
37      //create temporary mode
38      qNode<T>* deleteNode = nullptr;
39      deleteNode = (*frontPtr);
40
41      //add a check if queue is already empty
42      if((*frontPtr) == nullptr){
43          std::cout << "The queue is empty!" << std::endl;
44          return;
45      }
46
47      //check if the queue has only 1 node
48      if((*frontPtr) == (*backPtr)){
49          (*frontPtr) = nullptr;
50          (*backPtr) = nullptr;
51      }
52      else {
53          (*frontPtr) = (*frontPtr)->next;
54      }
55
56      //delete the node
57      deleteNode->next = nullptr;
58      delete deleteNode;
```


ILOB - [ILOB.dev] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
ILOB
ILOB modded.cp
queueLL.h
ILOB modded.cpp
queueLL.h
58 delete deleteNode;
59 }
60
61 template <typename T>
62 void displayAll(qNode<T>* frontPtr){
63 if (frontPtr == nullptr){
64 std::cout << "The queue is empty!" << std::endl;
65 return;
66 }
67 qNode<T>* current = frontPtr;
68 while (current != nullptr){
69 std::cout << current->data << " ";
70 current = current->next;
71 }
72 std::cout << std::endl;
73 }
74
75 template <typename T>
76 void checkfrontandback(qNode<T>* frontPtr, qNode<T>* backPtr){
77 if (frontPtr != nullptr){
78 std::cout << frontPtr->data << " ";
79 } else {
80 std::cout << "NULL ";
81 }
82 if (backPtr != nullptr){
83 std::cout << backPtr->data << std::endl;
84 } else {
85 std::cout << "NULL" << std::endl;
86 }
87 }
88
89 #endif

OUTPUT	
OBSERVATION	The code was changed to call check 'front' and 'back', and displayAll directly after every enqueue and dequeue operation. It now functions by printing a live trace of the front and back pointers alongside the full queue after every state change.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C

CODE

ILOC - [ILOC.dev] - [Executing] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project Classes Debug

ILOC
main.cpp

main.cpp

```cpp
#include <iostream>
#include <string>

class Job {
public:
    int id;
    std::string userName;
    int numPages;

    Job() : id(0), userName(""), numPages(0) {}
    Job(int jobId, std::string name, int pages) : id(jobId), userName(name), numPages(pages) {}
};

template <typename T>
class qNode {
public:
    T data;
    qNode<T>* next;
};

class Printer {
private:
    qNode<Job>* frontPtr;
    qNode<Job>* backPtr;
```

ILOC - [ILOC.dev] - [Executing] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project Classes Debug

ILOC
main.cpp

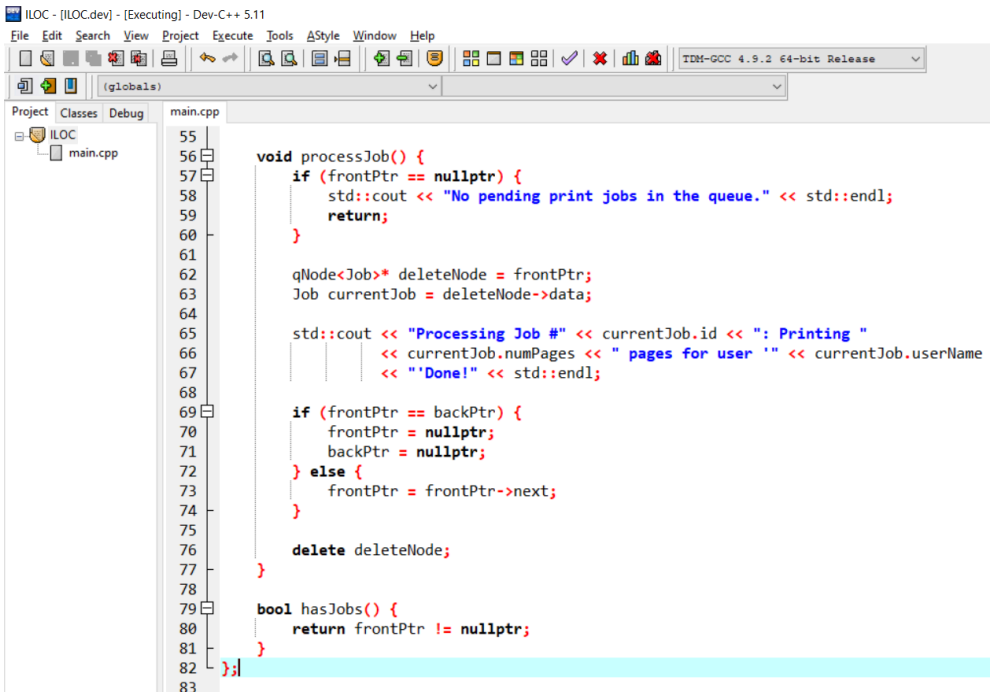
main.cpp

```cpp
public:
    Printer() {
        frontPtr = nullptr;
        backPtr = nullptr;
    }

    ~Printer() {
        while (frontPtr != nullptr) {
            qNode<Job>* temp = frontPtr;
            frontPtr = frontPtr->next;
            delete temp;
        }
    }

    void addJob(Job newJob) {
        qNode<Job>* newNode = new qNode<Job>;
        newNode->data = newJob;
        newNode->next = nullptr;

        if (frontPtr == nullptr) {
            frontPtr = newNode;
            backPtr = newNode;
        } else {
            backPtr->next = newNode;
            backPtr = newNode;
        }
        std::cout << "Added Job #" << newJob.id << " (" << newJob.numPages
                  << " pages) submitted by " << newJob.userName << "." << std::endl;
    }
```




```cpp
    void processJob() {
        if (frontPtr == nullptr) {
            std::cout << "No pending print jobs in the queue." << std::endl;
            return;
        }

        qNode<Job>* deleteNode = frontPtr;
        Job currentJob = deleteNode->data;

        std::cout << "Processing Job #" << currentJob.id << ": Printing "
                  << currentJob.numPages << " pages for user '" << currentJob.userName
                  << "'Done!" << std::endl;

        if (frontPtr == backPtr) {
            frontPtr = nullptr;
            backPtr = nullptr;
        } else {
            frontPtr = frontPtr->next;
        }

        delete deleteNode;
    }

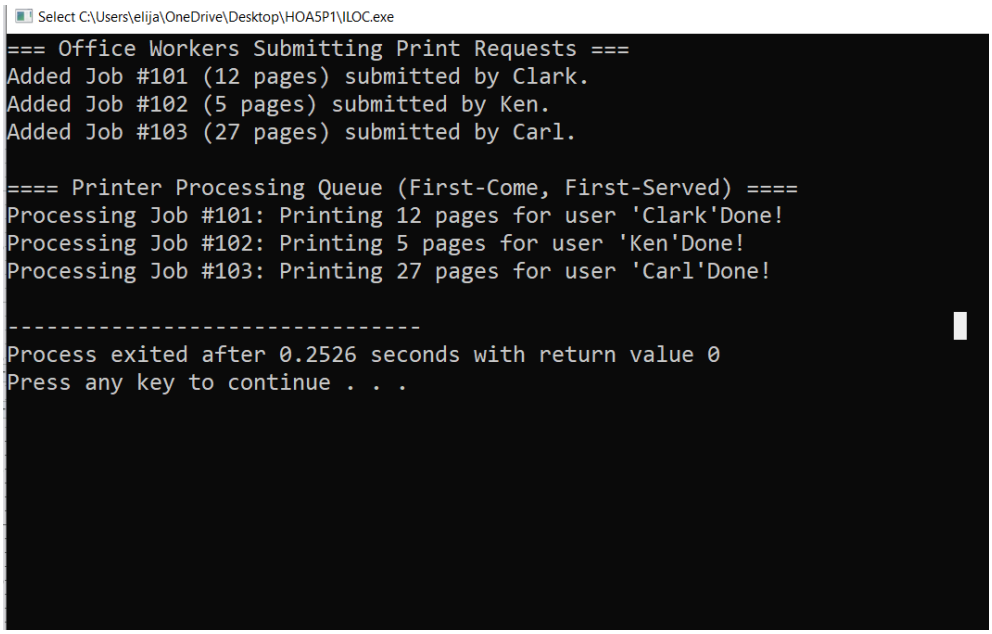
    bool hasJobs() {
        return frontPtr != nullptr;
    }
};
```

```cpp
int main() {
    Printer officePrinter;

    std::cout << "=== Office Workers Submitting Print Requests ===" << std::endl;
    officePrinter.addJob(Job(101, "Clark", 12));
    officePrinter.addJob(Job(102, "Ken", 5));
    officePrinter.addJob(Job(103, "Carl", 27));
    std::cout << std::endl;

    std::cout << "==== Printer Processing Queue (First-Come, First-Served) ====" << std::endl;
    while (officePrinter.hasJobs()) {
        officePrinter.processJob();
    }

    return 0;
}
```

OUTPUT	
OBSERVATION	Based on the code, creates a functional printer simulation using a custom link list queue. The 'job' class stores the details for each request that was made, while the 'printer' class controls the workflow.storing incoming jobs at the back of the queue and prints the one by one from the front, the run time logs that jobs were finished in the exact order they were received and memory is safely freed after each print task completed

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

“I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.”